Medicare: Smart Healthcare Management System with Prescription and Follow-Up Automation

Mini Project Documentation

---

1. Problem Statement

Many healthcare systems still depend on manual processes for handling appointments, prescriptions, follow-up scheduling, and patient record management. This leads to missed appointments, inefficient communication, delayed care, poor accessibility of medical history, and difficulty in verifying doctors. The proposed Smart Healthcare Management System solves these problems by digitizing the complete healthcare workflow through appointment booking, prescription upload, automated follow-up communication, medical history access, and doctor verification. This improves efficiency, accessibility, and the overall patient experience.

2. Introduction

The AI-Powered Smart Healthcare Assistant System is a digital healthcare platform designed to streamline patient-doctor interaction and healthcare service delivery. The system supports key functionalities such as appointment booking, doctor prescription upload, automated follow-up scheduling, medical history management, and doctor verification. It also integrates intelligent communication services and automation tools to improve patient engagement and healthcare efficiency. The platform ensures better accessibility, organized records, reduced manual effort, and timely follow-up care.

3. Objectives of the System

- To simplify appointment booking for patients.
- To enable secure prescription upload and follow-up scheduling.
- To provide access to past medical history and records.
- To automate reminder calls and notifications.
- To verify doctor credentials and maintain trust in the system.
- To improve overall healthcare workflow through digital automation.

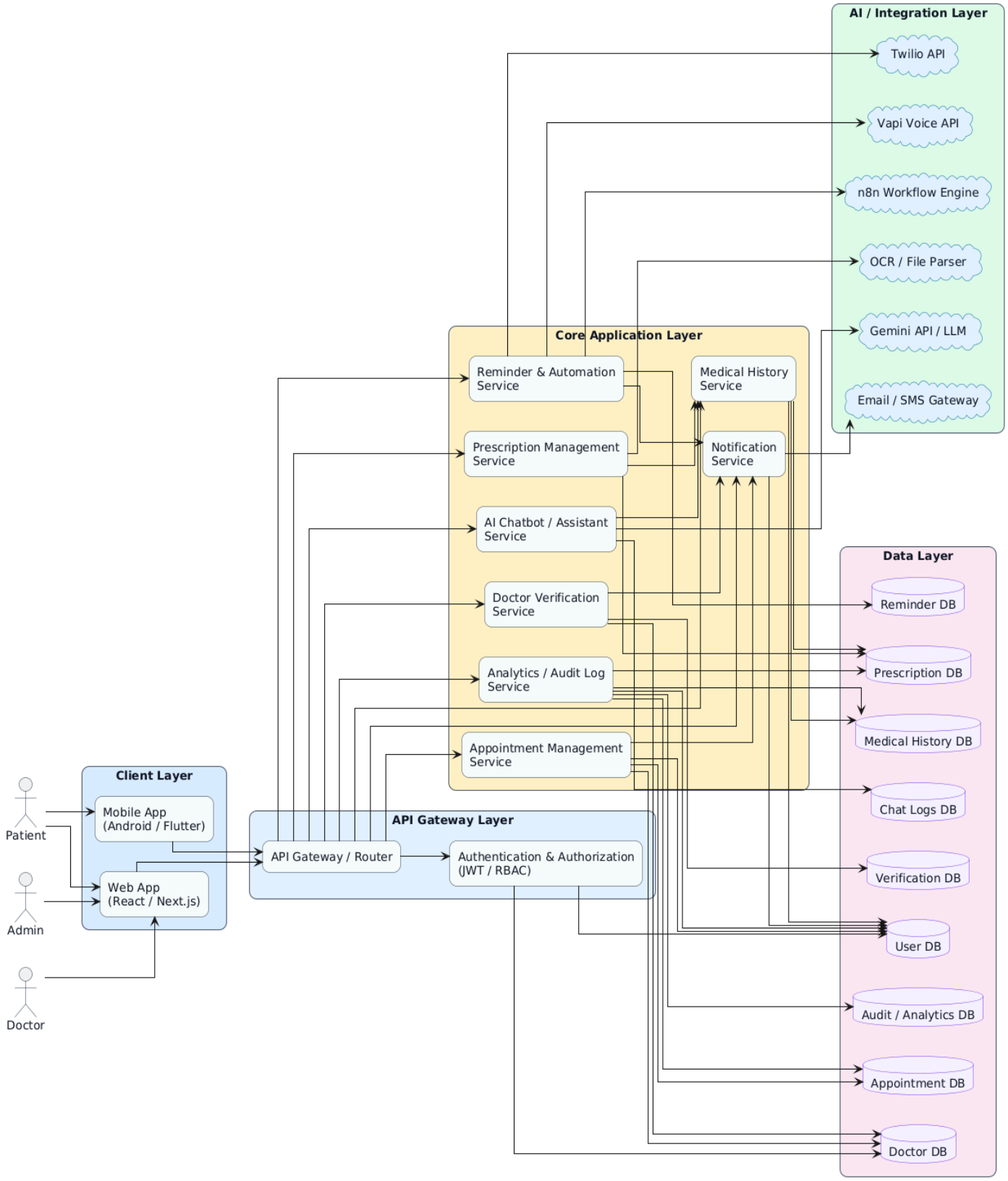
4. Scope of the System

The scope of the proposed system includes patient registration and login, doctor registration and verification, appointment scheduling, prescription management, follow-up reminders, medical history storage and retrieval, and intelligent chatbot-based interaction. The system is intended for healthcare organizations, clinics, and hospitals that require a centralized digital solution for patient management and communication.

5. System Architecture Diagram - Link

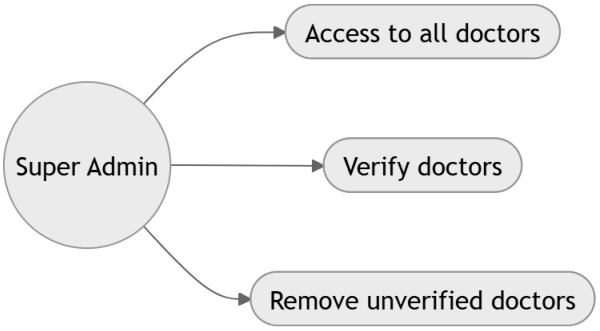
The system architecture diagram represents the overall structural design of the AI-Powered Smart Healthcare Assistant System. It illustrates how the client layer, API gateway, core application modules, service layer, data layer, and external integration layer work together. The architecture supports different user roles such as Patient, Doctor, and Admin through web and mobile interfaces. The backend services handle authentication, appointment management, prescription processing, medical history management, reminder automation, doctor verification, chatbot assistance, and notifications. The system also integrates with external services such as Twilio, Vapi, Gemini API, OCR parser, and workflow automation tools for communication, intelligence, and process automation. This diagram provides a complete high-level view of how the platform functions as an integrated healthcare solution.

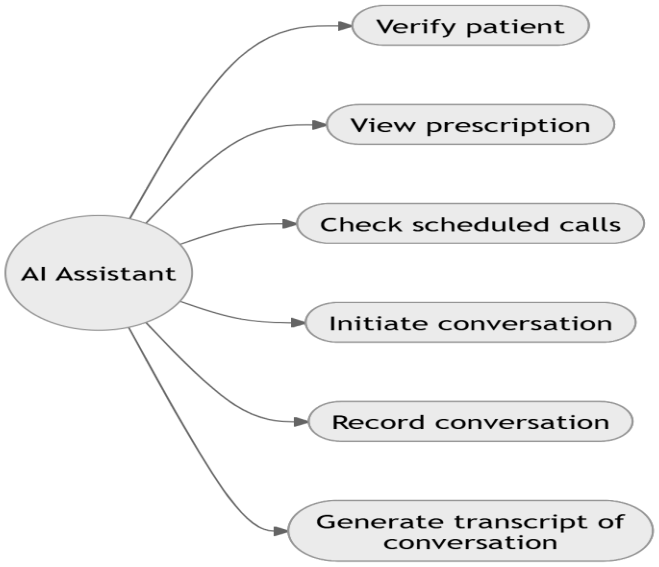
System Architecture Diagram - AI-Powered Smart Healthcare Assistant



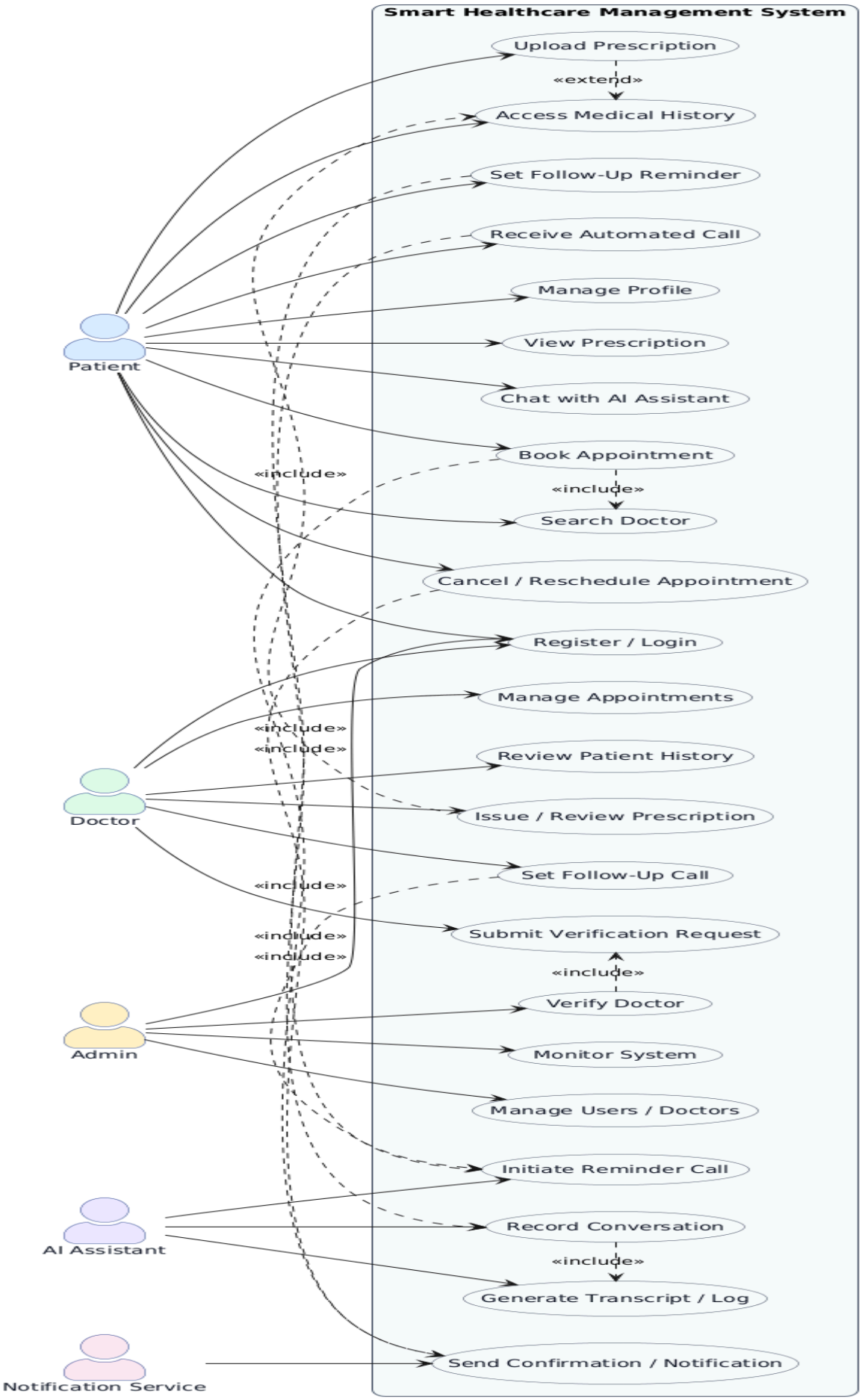
6. Use Case Diagram - Link

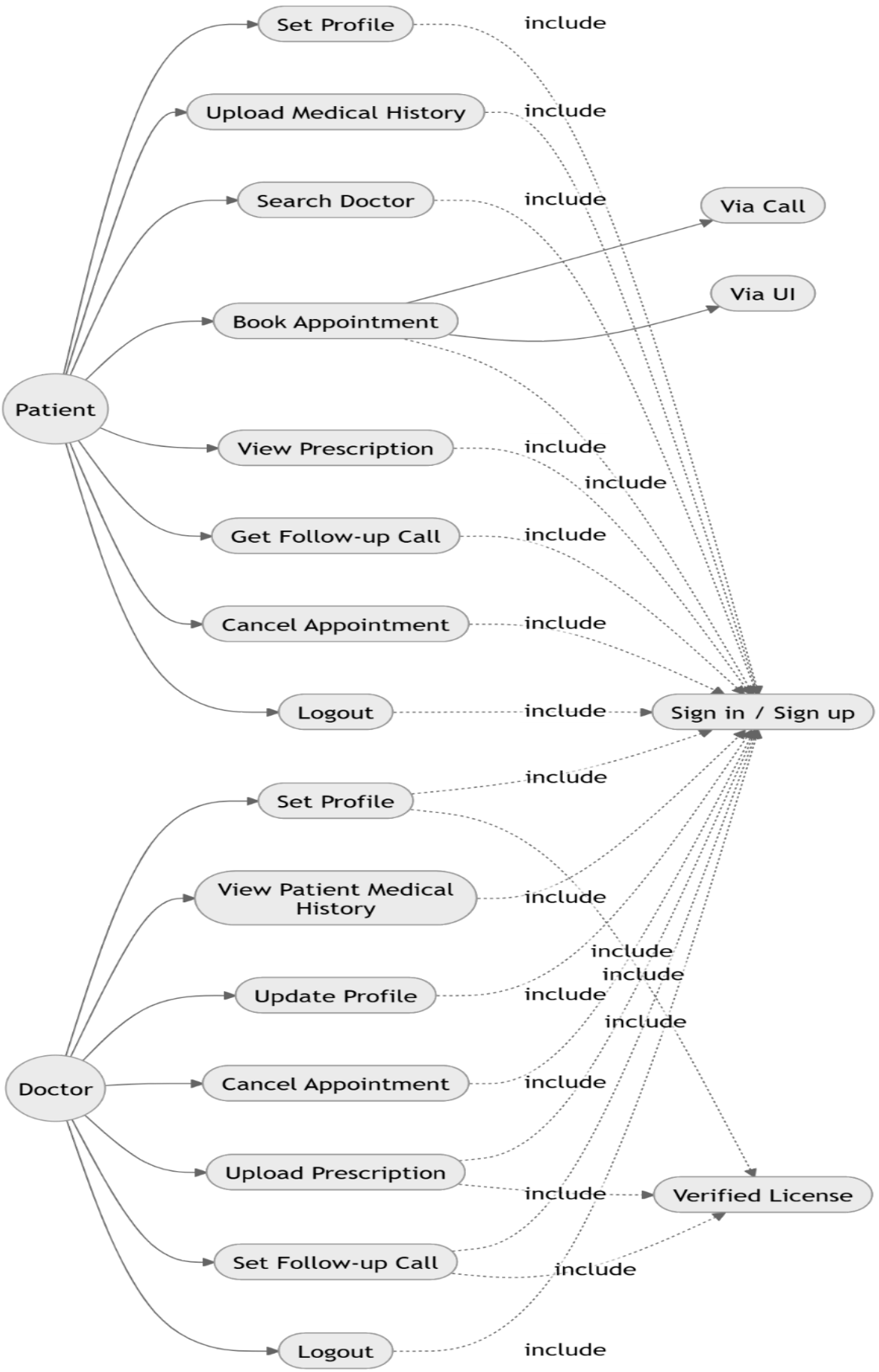
The use case diagram represents the functional interactions between the actors and the Smart Healthcare Management System. It identifies the major actors of the system, including Patient, Doctor, Admin/Super Admin, and AI Assistant, along with the key services they access. The patient can manage profile details, upload medical history, search for doctors, book and cancel appointments, view prescriptions, and receive follow-up calls. The doctor can view patient medical history, upload prescriptions, set follow-up calls, and manage appointments. The admin verifies doctors and manages the integrity of the platform. The AI Assistant supports verification, scheduled calls, conversation initiation, recording, and transcript generation. This diagram provides a clear overview of the functional requirements and user interaction boundaries of the proposed system.





Use Case Diagram - AI-Powered Smart Healthcare Assistant System




Patient
Set Profile
Upload Medical History
Search Doctor
Book Appointment
View Prescription
Get Follow-up Call
Cancel Appointment
Logout
Via Call
Via UI
Sign in / Sign up
Doctor
Set Profile
View Patient Medical History
Update Profile
Cancel Appointment
Upload Prescription
Set Follow-up Call
Logout
Verified License
include

7. Use Case Specifications

The following use case specifications provide the detailed functional flow of the system, including actors, goals, preconditions, triggers, normal scenario, exceptions, and open issues.

7.1 Book Appointment

USE-CASE:	**Book Appointment**
PRIMARY ACTOR:	Patient
GOAL IN CONTEXT:	To allow a patient to schedule an appointment with a doctor.
PRECONDITIONS:	1) The system is operational. 2) The patient is registered and logged into the system.
TRIGGER:	The patient chooses the option to book an appointment.
SCENARIO:	1) Patient opens the healthcare application. 2) Patient selects the doctor and available time slot. 3) Patient confirms appointment details. 4) System schedules the appointment through a phone call or UI and sends confirmation.
EXCEPTIONS:	1) If the selected slot is unavailable, system asks patient to choose another slot. 2) If system error occurs, appointment is not booked and an error message is shown.
PRIORITY:	Essential, must be implemented.
WHEN AVAILABLE:	First increment.
FREQUENCY OF USE:	Frequently.
CHANNEL TO ACTOR:	Web application interface.
SECONDARY ACTOR:	Doctor
CHANNELS TO SECONDARY ACTORS:	Notification through system dashboard.

OPEN ISSUES:	Handling appointment conflicts and cancellations.

7.2 Upload Prescription and Set Follow-Up

USE-CASE:	**Upload Prescription & Set Follow-Up**
PRIMARY ACTOR:	Doctor
GOAL IN CONTEXT:	To allow the doctor to upload a prescription after consultation and schedule a follow-up appointment for the patient.
PRECONDITIONS:	1) Doctor is registered and logged into the system. 2) Patient consultation is completed. 3) Patient record exists in the system.
TRIGGER:	Doctor selects the patient after consultation and chooses the option to upload prescription.
SCENARIO:	1) Doctor logs into the healthcare system. 2) Doctor selects the patient from appointment list. 3) Doctor uploads or writes prescription details 4) Doctor sets follow-up date and time (if required). 5) System saves prescription securely. 6) System notifies the patient about prescription and follow-up schedule.
EXCEPTIONS:	1) If file upload fails, system asks doctor to retry. 2) If follow-up slot is unavailable, system prompts doctor to choose another date. 3) If system error occurs, prescription is not saved and error message is displayed.
PRIORITY:	Essential.
WHEN AVAILABLE:	First increment.

FREQUENCY OF USE:	Frequently.
CHANNEL TO ACTOR:	Doctor dashboard (Web/Mobile).
SECONDARY ACTOR:	Patient
CHANNELS TO SECONDARY ACTORS:	Mobile app notification / Email / SMS.
OPEN ISSUES:	Secure storage of prescriptions, integration with notification services, follow-up rescheduling management.

7.3 Follow Automation Call

USE-CASE:	**Follow Automation Call**
PRIMARY ACTOR:	System
GOAL IN CONTEXT:	To automatically call or remind patients about Health Conditions after visit.
PRECONDITIONS:	1) Patient contact details are stored. 2) The reminder schedule is configured.
TRIGGER:	Scheduled time for reminder arrives.
SCENARIO:	1) System detects reminder schedule. 2) System initiates automated call or notification. 3) Patient receives reminder message or call. 4) System logs reminder activity.
EXCEPTIONS:	1) If call fails, system retries later. 2) If contact number is invalid, reminder is not delivered.
PRIORITY:	Medium priority.
WHEN AVAILABLE:	Second increment.

FREQUENCY OF USE:	Regularly.
CHANNEL TO ACTOR:	Automated call or notification system.
SECONDARY ACTOR:	Patient
CHANNELS TO SECONDARY ACTORS:	Phone call or mobile notification.
OPEN ISSUES:	Integration with telecommunication service APIs.

7.4 Medical History Access

USE-CASE:	**Medical History Access**
PRIMARY ACTOR:	Patient, Doctor
GOAL IN CONTEXT:	To allow patients and doctors to view past medical records and prescriptions.
PRECONDITIONS:	1) The patient is registered in the system. 2) Medical records exist in the database.
TRIGGER:	User selects the medical history option.
SCENARIO:	1) Patient, Doctor logs into the system. 2) Patient, Doctor navigates to the medical history section. 3) System retrieves stored records. 4) Medical history is displayed to users.
EXCEPTIONS:	1) If no records exist, system shows “No records found”. 2) If database error occurs, system shows error message.
PRIORITY:	High priority.
WHEN AVAILABLE:	First increment.
FREQUENCY OF USE:	Frequently.
CHANNEL TO ACTOR:	Web or mobile interface.
SECONDARY ACTOR:	NA

CHANNELS TO SECONDARY ACTORS:	NA
OPEN ISSUES:	Data security and patient privacy compliance.

7.5 Doctor Verification

USE-CASE:	**Doctor Verification**
PRIMARY ACTOR:	System Administrator
GOAL IN CONTEXT:	To verify the credentials and authenticity of registered doctors.
PRECONDITIONS:	1) Doctor registration information is submitted. 2) Verification documents are uploaded.
TRIGGER:	Administrator initiates verification process.
SCENARIO:	1) Admin reviews doctor details and uploaded documents. 2) Admin verifies credentials and medical license. 3) System updates doctor status as verified 4) Doctor receives verification confirmation.
EXCEPTIONS:	1) If documents are invalid, verification is rejected. 2) Admin requests additional documents if required.
PRIORITY:	Essential.
WHEN AVAILABLE:	First increment.
FREQUENCY OF USE:	Occasionally.
CHANNEL TO ACTOR:	Admin dashboard.
SECONDARY ACTOR:	Doctor
CHANNELS TO SECONDARY ACTORS:	Email or system notification.
OPEN ISSUES:	Validation process for medical licenses and certificates.

7.6 Cancel or Reschedule Appointment

USE-CASE:	**Cancel or Reschedule Appointment**
PRIMARY ACTOR:	Patient
GOAL IN CONTEXT:	To allow the patient to cancel or reschedule an already booked appointment.
PRECONDITIONS:	1) Patient is registered and logged into the system. 2) An existing appointment is already scheduled.
TRIGGER:	Patient selects the option to manage existing appointments.
SCENARIO:	1) Patient logs into the system. 2) Patient navigates to "My Appointments." 3) Patient selects the appointment to cancel or reschedule. 4) If rescheduling, patient selects a new available date and time. 5) System updates appointment details. 6) System sends confirmation notification to both patient and doctor.
EXCEPTIONS:	1) If no slots are available, system asks patient to choose another date. 2) If cancellation deadline has passed, system restricts cancellation and displays message. 3) If system error occurs, changes are not saved.
PRIORITY:	High priority.
WHEN AVAILABLE:	First increment.
FREQUENCY OF USE:	Frequently.
CHANNEL TO ACTOR:	Mobile or Web application.
SECONDARY ACTOR:	Doctor

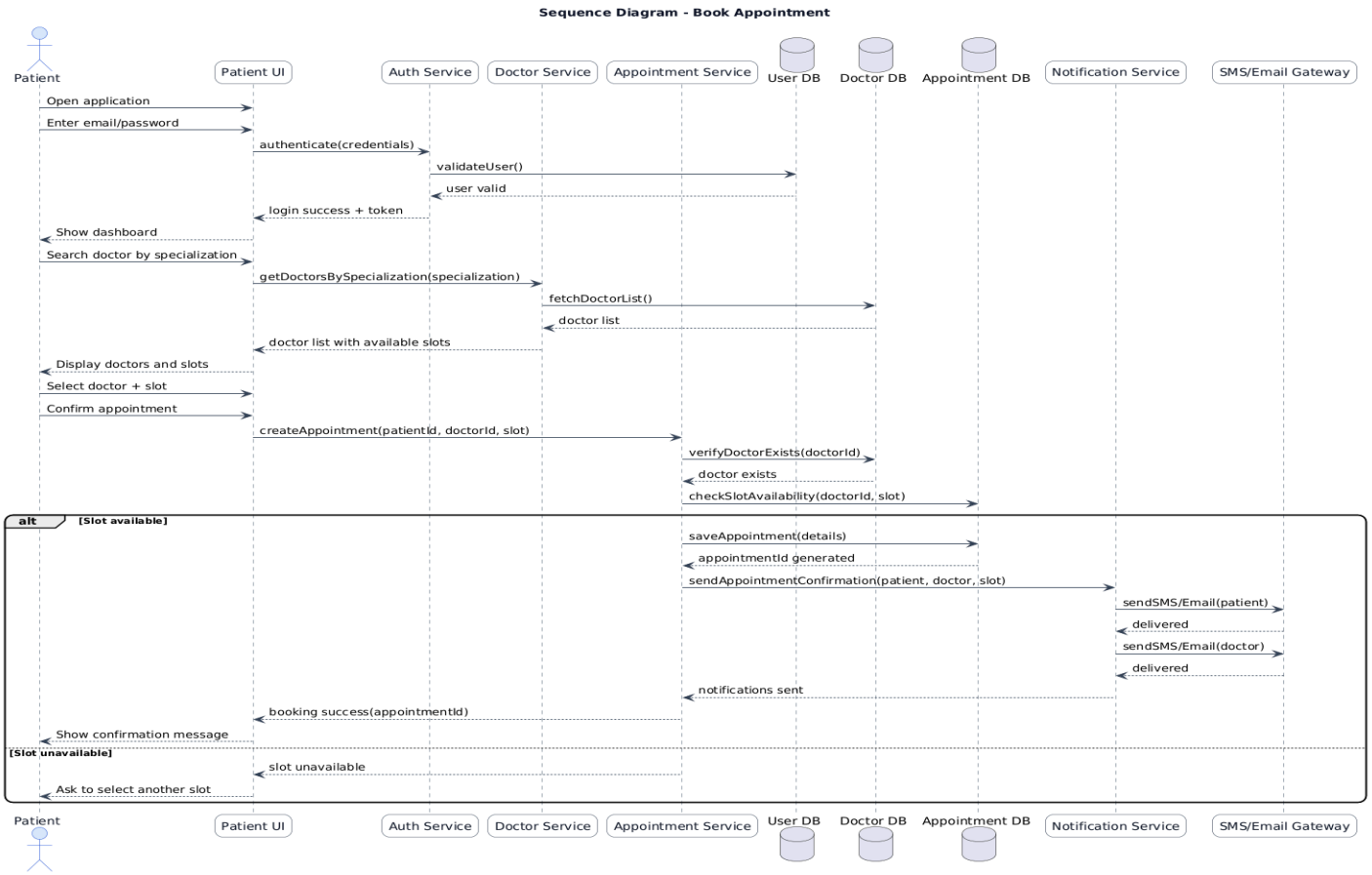
CHANNELS TO SECONDARY ACTORS:	Doctor dashboard notification / Email.
OPEN ISSUES:	Cancellation policy rules, refund handling (if payment integrated), and real-time slot synchronization.

8. Sequence Diagram - Link

The sequence diagram represents the time-ordered interaction between system actors, interfaces, backend services, and databases during the execution of a use case. It shows how requests are initiated, validated, processed, and completed in a sequential manner. In this system, the sequence diagrams illustrate workflows such as book appointment, doctor verification, prescription upload, follow-up call setup, and AI assistant interaction. These diagrams help in understanding the order of communication between patient, doctor, admin, system modules, and supporting services. They also highlight validations, confirmations, and notifications generated during the process.

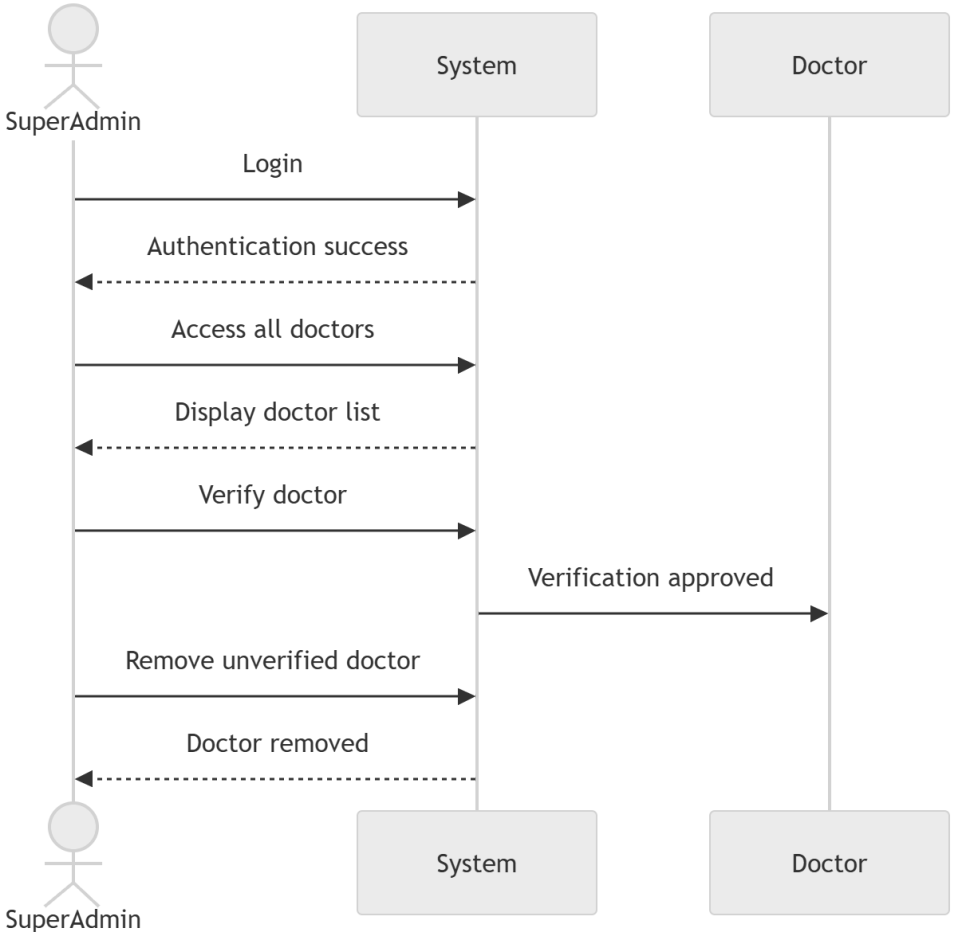
8.1 Complete Sequence Diagram – Book Appointment

The complete sequence diagram for book appointment illustrates the entire appointment booking process from patient login to appointment confirmation. The patient accesses the dashboard, searches for doctors, selects a preferred doctor and slot, and confirms the booking request. The appointment service checks doctor existence and slot availability through the relevant databases. If the slot is available, the appointment is created and stored in the appointment database. Finally, the notification service sends confirmation messages to both patient and doctor through email or SMS gateway. This diagram clearly shows the step-by-step interaction among frontend, services, databases, and notification systems.



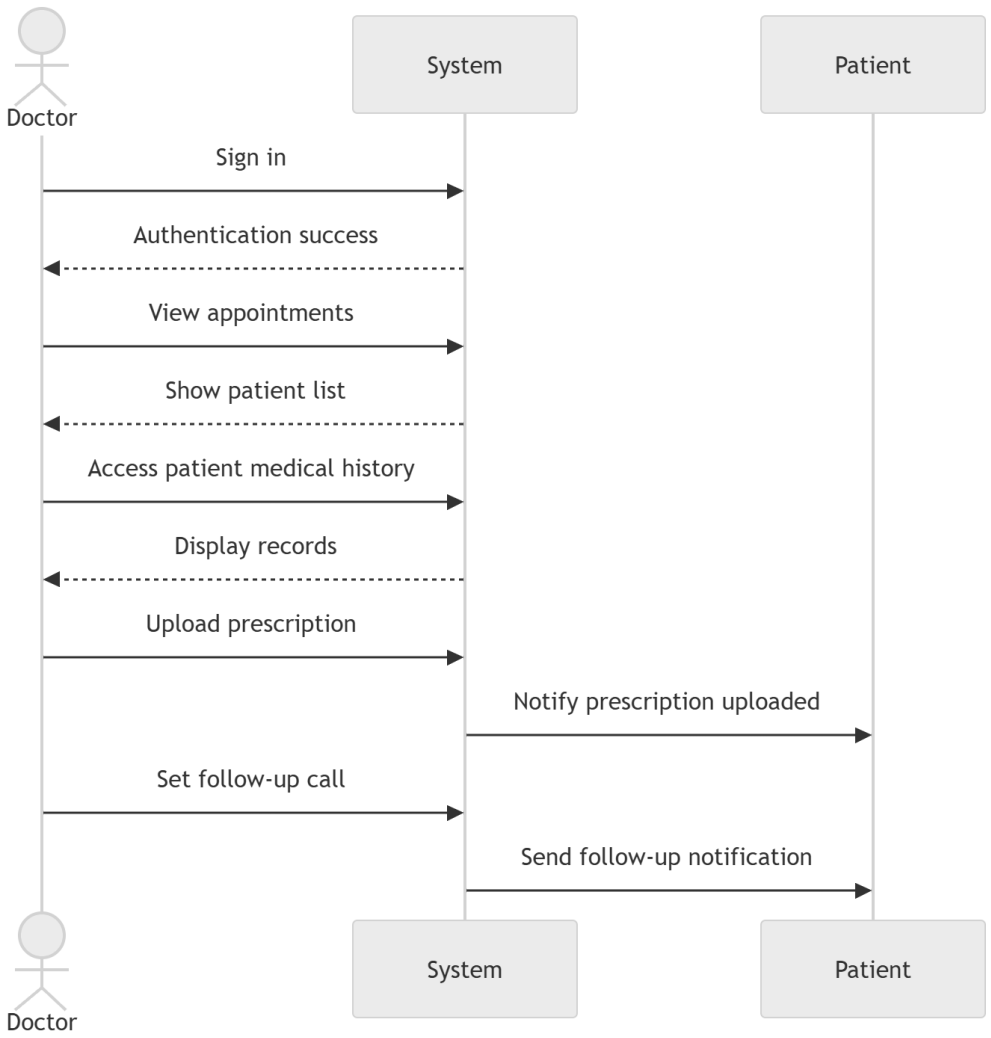
8.2 Super Admin Sequence Diagram

The Super Admin sequence diagram shows how the administrator authenticates into the system, views the list of doctors, verifies eligible doctors, and removes unverified or invalid doctors. It highlights the interaction between the administrator, the system, and doctor verification status update mechanism.



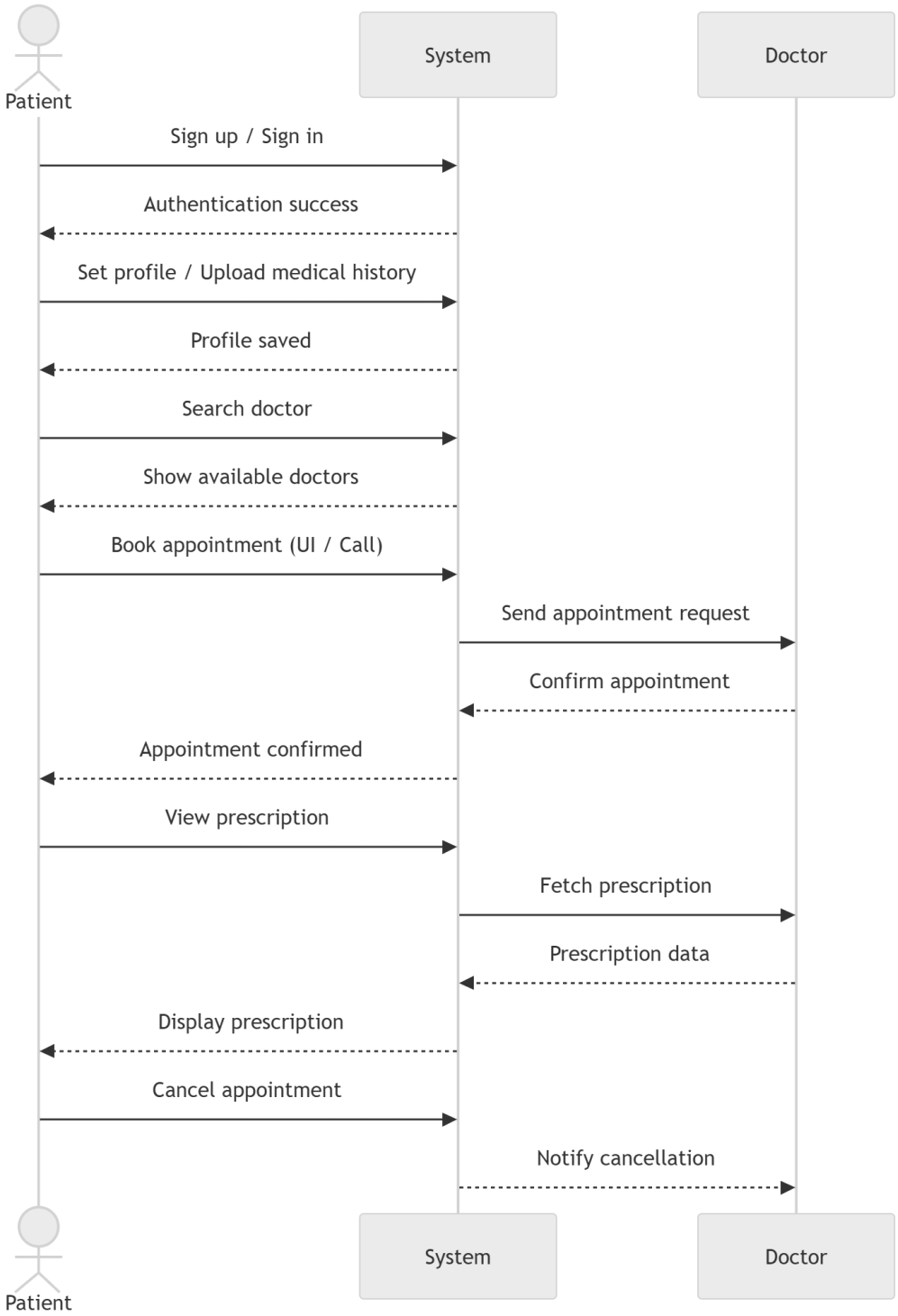
8.3 Doctor Sequence Diagram

The Doctor sequence diagram represents the doctor-side workflow in the system. After login, the doctor can view appointments, access patient medical history, upload prescription details, and set follow-up calls. The system then notifies the patient regarding the uploaded prescription and scheduled follow-up.



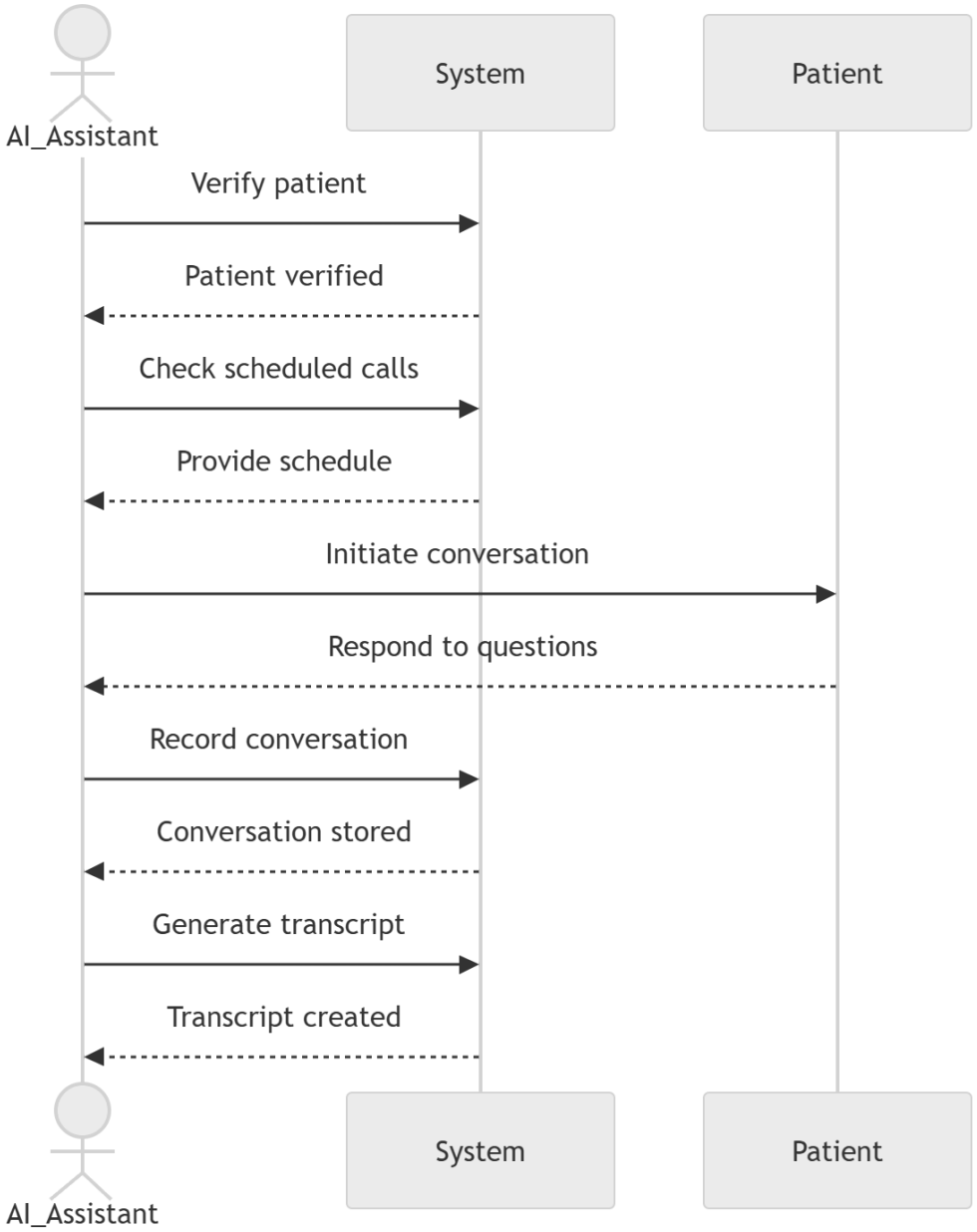
8.4 Patient Sequence Diagram

The Patient sequence diagram shows how a patient signs up or signs in, manages profile information, uploads medical history, searches for doctors, books appointments, views prescriptions, and cancels appointments. It represents the direct interaction between patient, system, and doctor for service delivery.



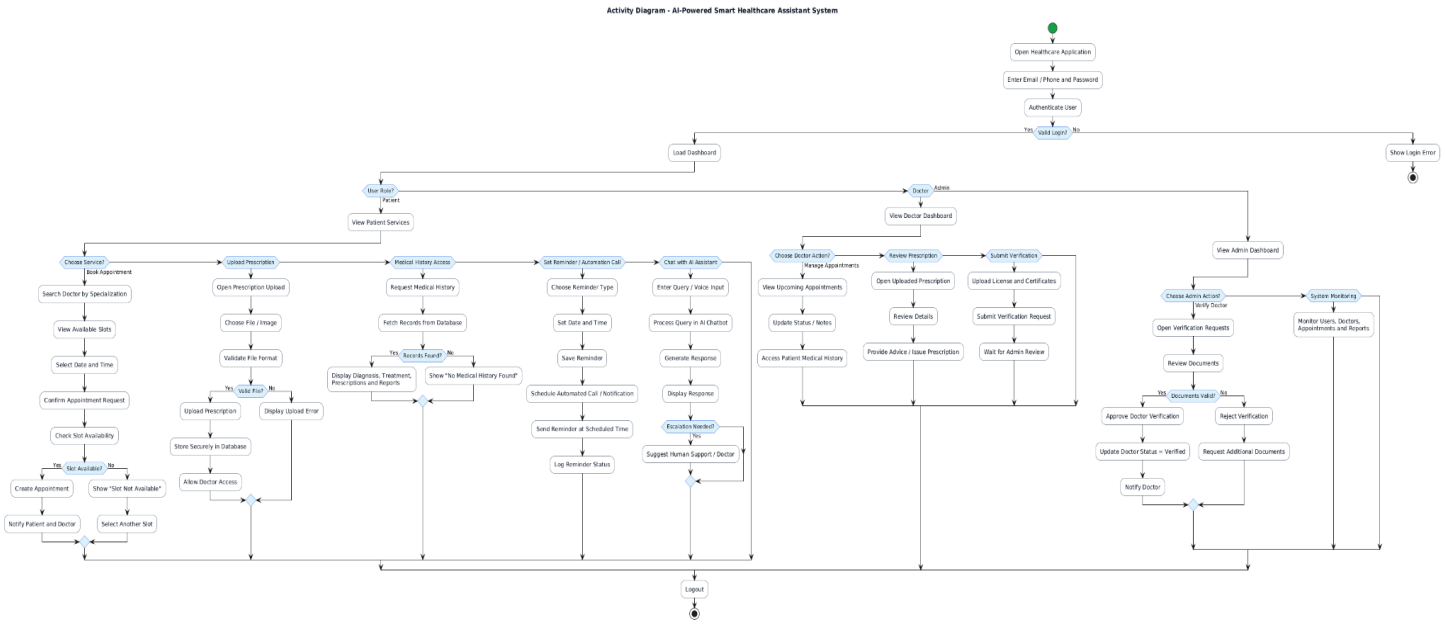
8.5 AI Assistant Sequence Diagram

The AI Assistant sequence diagram represents the flow of automated patient communication. It includes patient verification, checking scheduled calls, initiating conversation, responding to patient queries, recording the conversation, and generating a transcript. This sequence demonstrates how the AI assistant automates patient follow-up and communication support.


AI_Assistant
System
Patient
Verify patient
Patient verified
Check scheduled calls
Provide schedule
Initiate conversation
Respond to questions
Record conversation
Conversation stored
Generate transcript
Transcript created
AI_Assistant
System
Patient

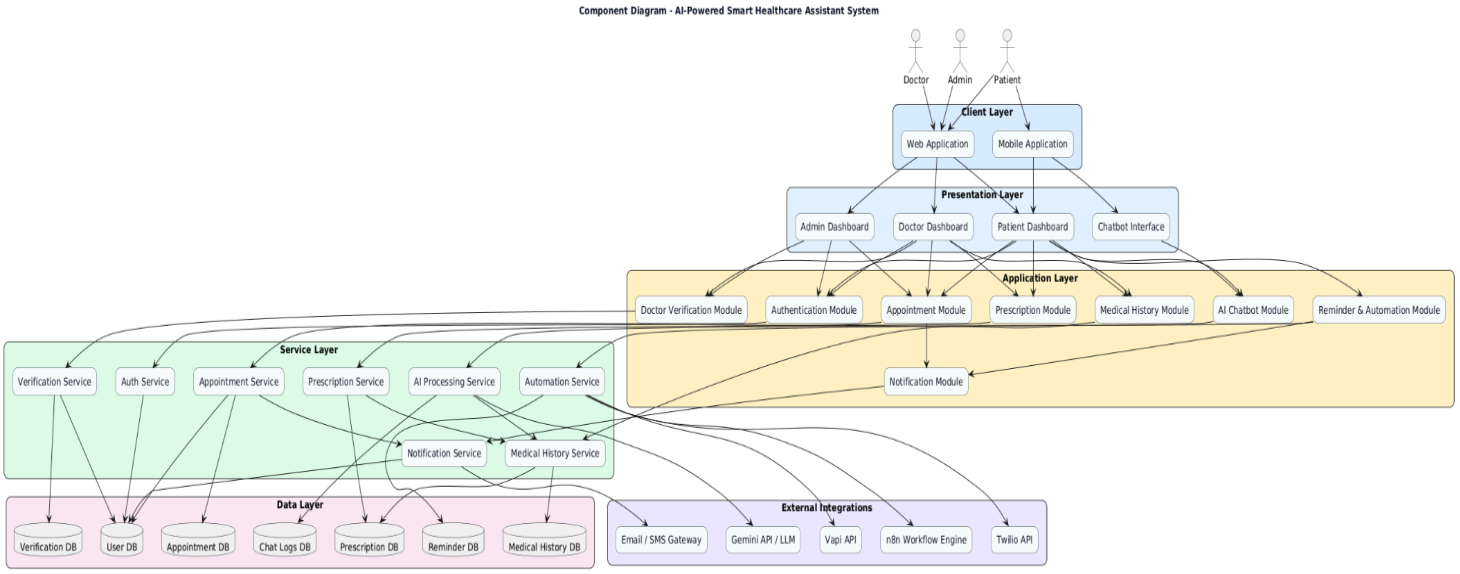
9. Activity Diagram - Link

The activity diagram represents the dynamic workflow of the AI-Powered Smart Healthcare Assistant System. It starts with user authentication and then branches according to the role of the logged-in user, such as Patient, Doctor, or Admin. The patient can perform actions like appointment booking, prescription viewing, medical history access, reminder setting, and chatbot interaction. The doctor can manage appointments, review patient records, upload prescriptions, and schedule follow-up communication. The admin performs doctor verification and system monitoring tasks. This diagram captures the flow of activities, decisions, validations, and system responses in an organized manner.



10. Component Diagram - Link

The component diagram represents the high-level modular architecture of the AI-Powered Smart Healthcare Assistant System. It shows the interaction among the client layer, presentation layer, application modules, backend services, databases, and external APIs. Users interact with the system through web and mobile applications connected to dashboards and chatbot interfaces. These interfaces communicate with core modules such as authentication, appointment management, prescription management, medical history, reminders, doctor verification, AI chatbot, and notifications. The service layer processes business logic and communicates with data storage and third-party APIs such as Twilio, Vapi, Gemini API, and workflow engines. This diagram emphasizes modularity, integration, and separation of concerns in the system design.

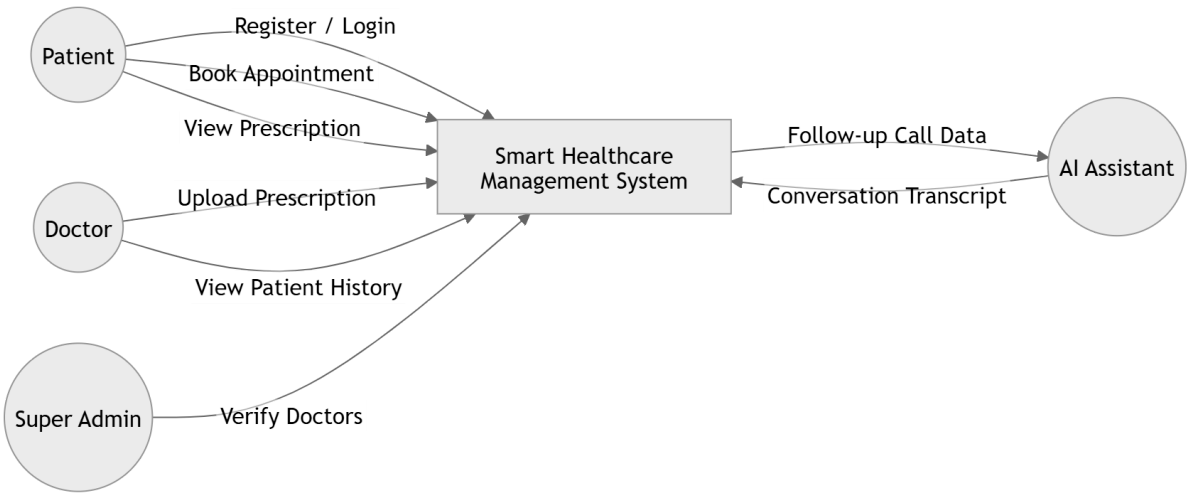


11. Data Flow Diagram

The Data Flow Diagram represents how data moves between external entities, processes, and data stores in the Smart Healthcare Management System. It helps in understanding the transformation and storage of information at different levels of abstraction.

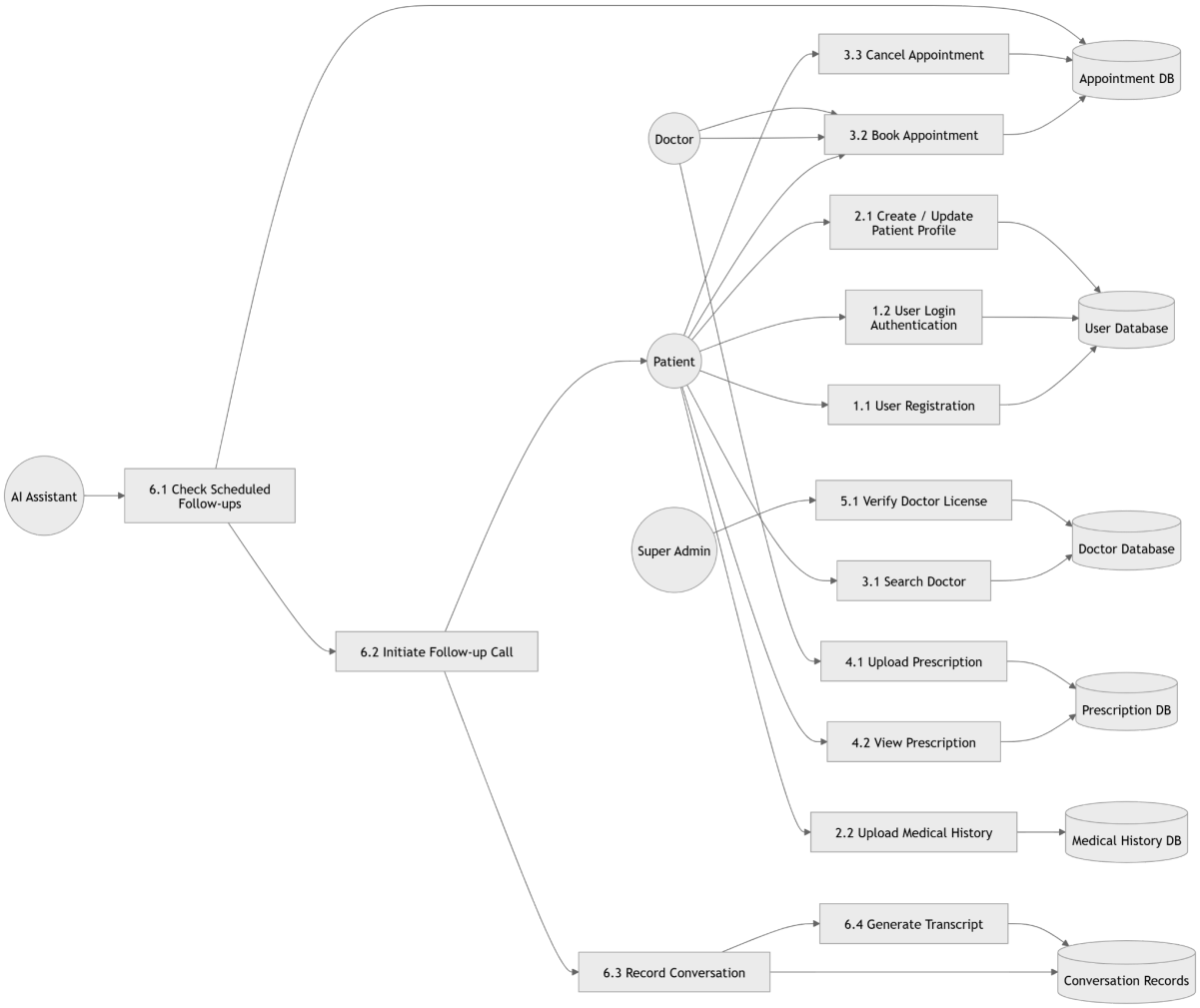
11.1 DFD Level 0

The Level 0 DFD, also known as the context diagram, provides a high-level representation of the Smart Healthcare Management System as a single process interacting with external entities such as Patient, Doctor, Super Admin, and AI Assistant. It shows how users provide login details, appointment requests, prescriptions, and verification inputs to the system, while the system returns confirmations, prescription data, follow-up data, and transcripts. This diagram helps in understanding the external boundaries and primary data exchanges of the system.



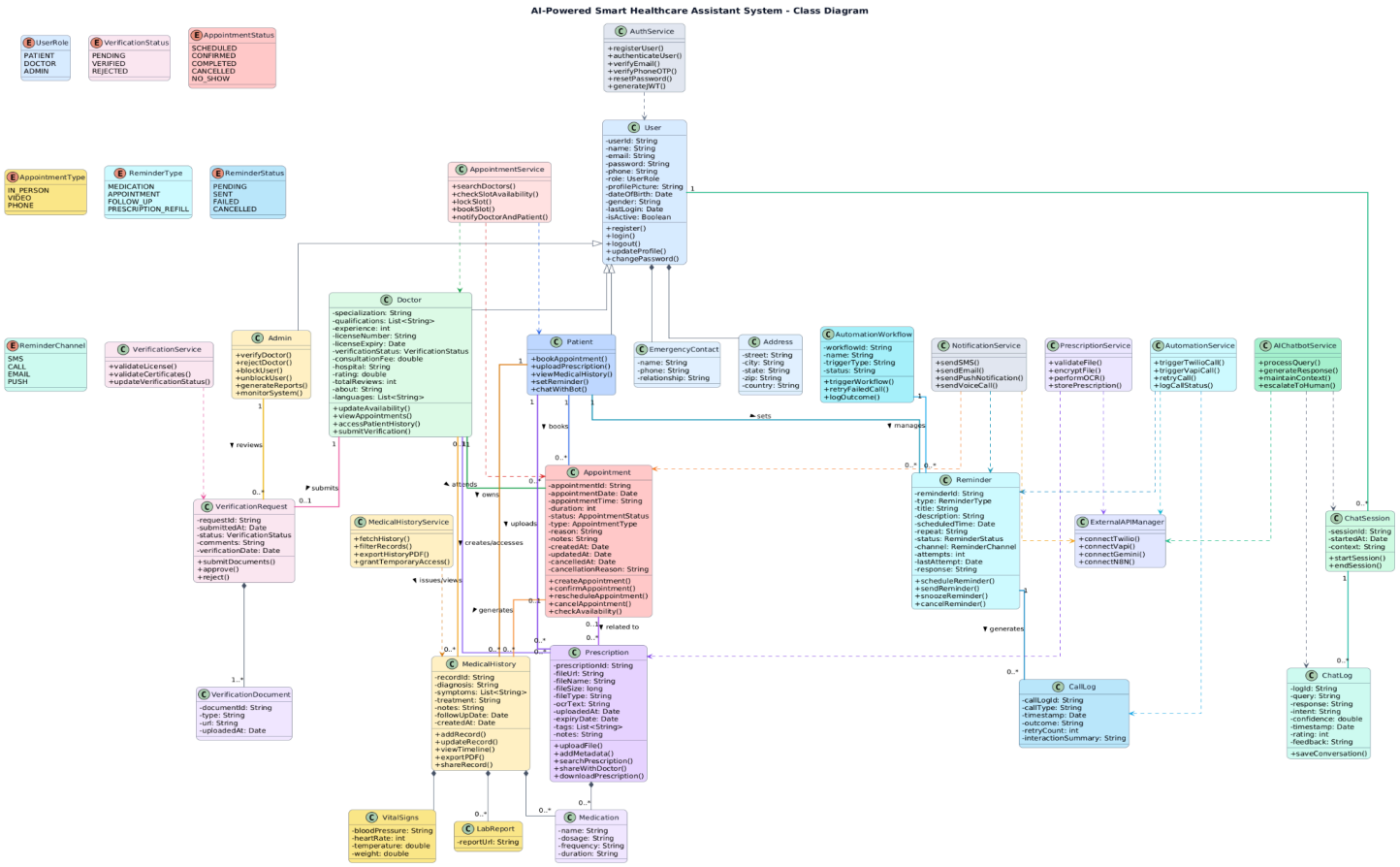
11.2 DFD Level 1

The Level 1 DFD expands the main system process into detailed sub-processes such as book appointment, upload prescription, view prescription, upload medical history, verify doctor license, record conversation, and schedule follow-up calls. It also illustrates the associated data stores such as user database, appointment database, doctor database, prescription database, medical history database, and conversation records. This diagram provides a more detailed understanding of internal data movement and processing logic.



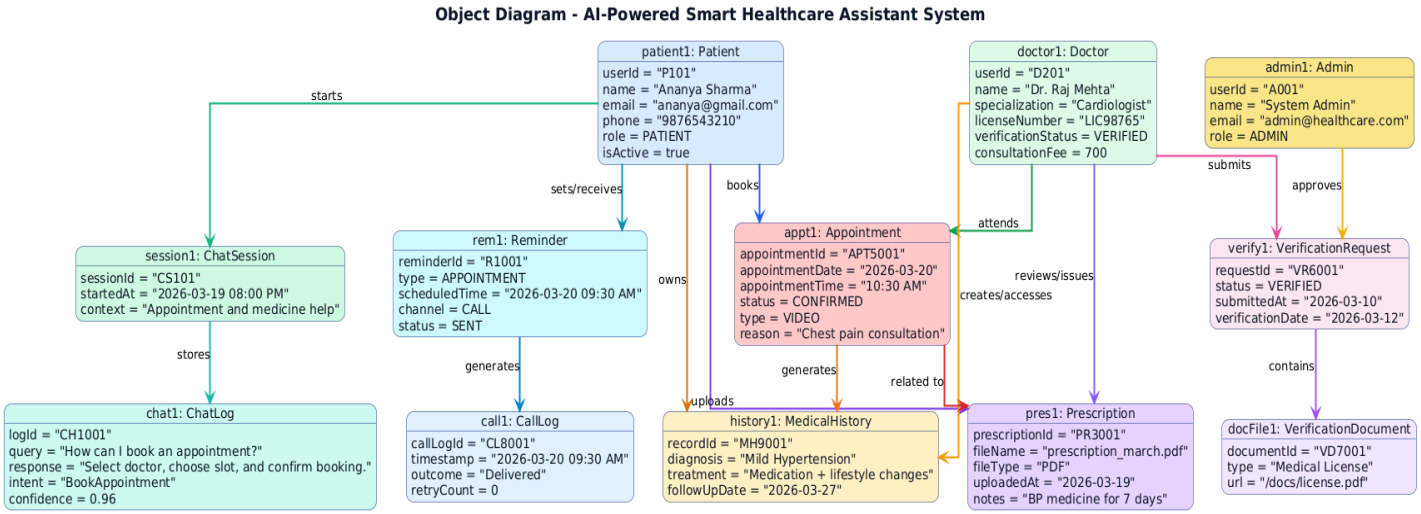
12. Class Diagram- Link

The class diagram represents the static structural design of the AI-Powered Smart Healthcare Assistant System. It identifies the major classes of the system such as User, Patient, Doctor, Admin, Appointment, Prescription, MedicalHistory, Reminder, VerificationRequest, ChatSession, and related service classes. The diagram shows the attributes, methods, inheritance relationships, associations, and dependencies among these classes. It forms the foundation for the object-oriented design of the system and helps in understanding how system entities are organized and related to one another.



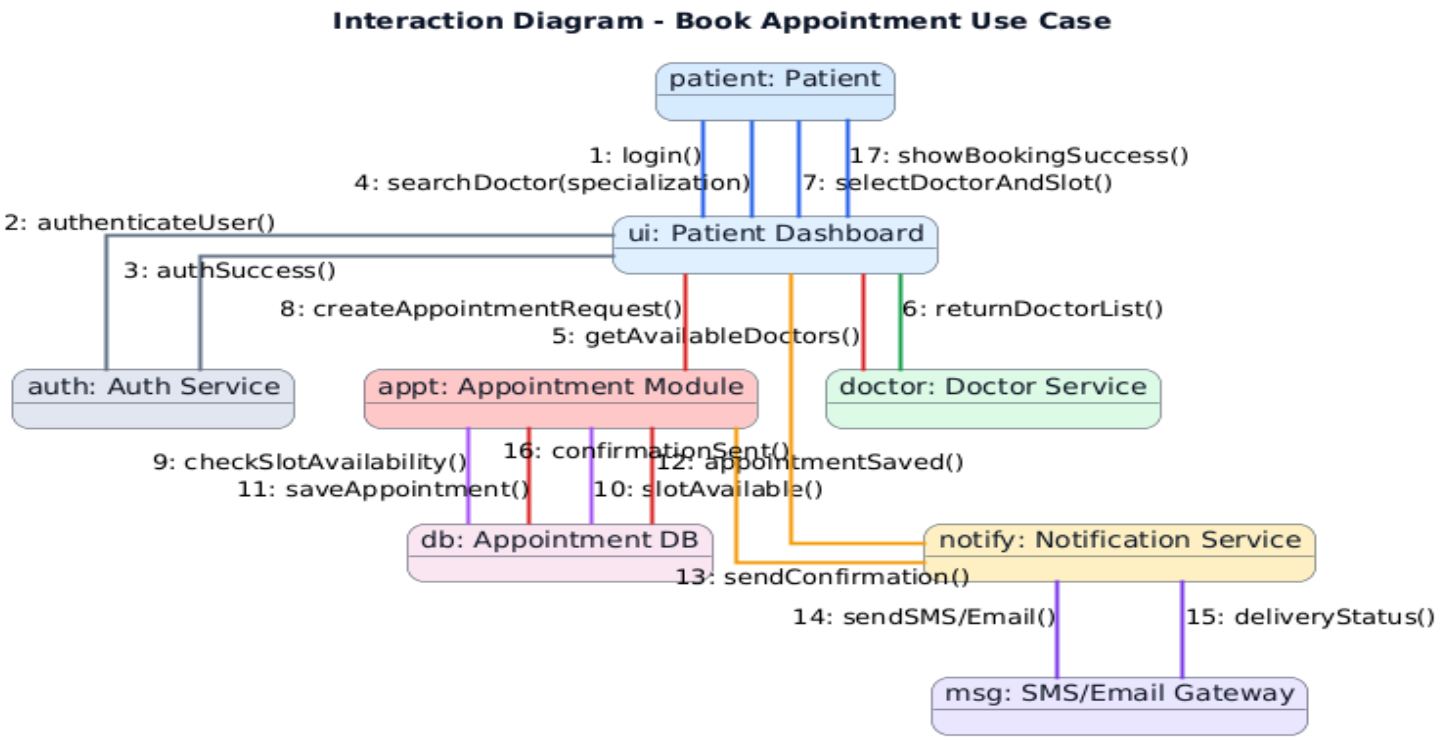
13. Object Diagram- Link

The object diagram represents a snapshot of the AI-Powered Smart Healthcare Assistant System at a specific moment in time. It illustrates real instances of objects such as patient, doctor, appointment, prescription, medical history, reminder, call log, verification request, and chatbot session with sample attribute values. This diagram helps in visualizing how actual system objects are linked during execution and how the class structure is instantiated in practice.



14. Interaction Diagram- Link

The interaction diagram, also known as the communication diagram, represents how objects in the system collaborate through message passing to complete a specific use case. In the Smart Healthcare Management System, the interaction diagram for appointment booking shows the communication between Patient, Dashboard, Authentication Service, Doctor Service, Appointment Module, Appointment Database, Notification Service, and SMS/Email Gateway. It focuses on the structural organization of collaborating objects and the order of messages exchanged among them.



15. Conclusion

The AI-Powered Smart Healthcare Assistant System provides a comprehensive digital solution for modern healthcare workflow management. It simplifies appointment scheduling, supports secure prescription handling, enables automated follow-up communication, provides accessible medical history records, and ensures doctor authenticity through verification. By integrating automation, structured data management, and intelligent communication services, the system reduces manual effort and improves patient care efficiency. The design models and diagrams presented in this document collectively describe the functional, structural, and behavioral aspects of the system, making it suitable for implementation as a robust healthcare support platform.

16. Future Scope

The future scope of the system includes integration with electronic health record platforms, multilingual AI voice assistance, predictive analytics for patient follow-up, smart prescription interpretation using advanced OCR and NLP, video consultation integration, wearable device data synchronization, and cloud-based scalability for large healthcare organizations. These enhancements can further improve patient experience, healthcare reach, and operational efficiency.

17. References

1. Software Engineering Mini Project document and SRS provided for the project.
2. UML design artifacts prepared for the AI-Powered Smart Healthcare Assistant System.
3. Standard software engineering concepts for use case, sequence, activity, component, class, object, interaction, and data flow diagrams.